

ARTI403 – Lab 5

Spatial Filtering – Smoothing and Sharpening

Name: Ali Jafar Altaweel

ID: 2240006456

Section: 7MA2

GitHub: <https://github.com/Ali-developer/Image-Processing/tree/main/Lab5>

In this lab, I worked on spatial filtering, where each output pixel is computed from a small neighborhood of the input image instead of a single pixel. I used two families of filters: smoothing filters, which average the neighborhood and blur the image, and sharpening filters, which emphasize the differences between neighboring pixels. The main objective was to understand how the size and the weights of a kernel decide the result.

Lab Tasks

Task 1 – Box filter

Convolve an image with a 7x7 box filter, and output the original and smoothed image side by side.

Task 2 – Gaussian filters

Using an image of your choice, apply a 5x5 Gaussian filter to the image, then a 21x21 Gaussian filter, and show the three images side by side.

Task 3 – Laplacian sharpening

Using an image of your choice, apply a 3x3 Laplacian filter to sharpen the image, following these steps: read the image, convert it to float32, use the `cv2.Laplacian()` function, sharpen the image using `np.clip(image_float - laplacian, 0, 1)`, and plot the original image, the resulting image after applying the Laplacian filter, and the sharpened image.

Images Used

For the unsharp masking and the box filter I used `Parrot.png`, the 453 × 340 color photograph of a macaw that the manual refers to. For the Gaussian filters I chose `Chelsea the cat`, a 451 × 300 color image whose fur has a lot of fine detail, and for the Laplacian I chose the cameraman, a 512 × 512 grayscale image with clear edges. The last two come from `skimage.data`.

All three images are saved in the `images` folder of this lab, and the notebook reads them from there with `cv2.imread()`. Since OpenCV loads color images as BGR, I converted them to RGB before displaying them with Matplotlib.

Unsharp Masking

I started with the code of the manual. The image is converted to float32 between 0 and 1 and blurred with a Gaussian of sigma 2, then the difference between the image and its blurred version is multiplied by an amount of 1.5 and added back to the image.

That difference is the mask. When I displayed it, it was gray almost everywhere and showed only the edges of the parrot, the eye and the feathers. Adding it back makes exactly those places stronger.

To put a number on the result I measured the variance of the Laplacian, which grows with the amount of fine detail. It was 442 for the original, 4 for the blurred image and 2278 for the sharpened one, about five times the original.

I also tried other amounts:

- With 0.5 the sharpening was mild, and less than 1% of the values had to be clipped
- With 1.5, the value of the manual, 1.9% were clipped
- With 4 the image looked harsh, with visible halos around the edges, and 5.2% were clipped



Figure 1 – The manual's unsharp masking: original, blurred and sharpened



Figure 2 – The mask, and the effect of the amount parameter

Box Filter

For the first task I built a 7×7 kernel in which each of the 49 weights is $1/49$, about 0.0204, so that they sum to 1. I convolved the parrot with it using `cv2.filter2D()` and displayed the original and the smoothed image side by side.

The smoothed image lost the texture of the feathers and every edge became a soft ramp. The variance of the Laplacian fell from 442 to 4.5, while the mean stayed at 106.1, because the weights sum to 1. As a check, I compared my result with `cv2.blur()` and the two were identical pixel by pixel.

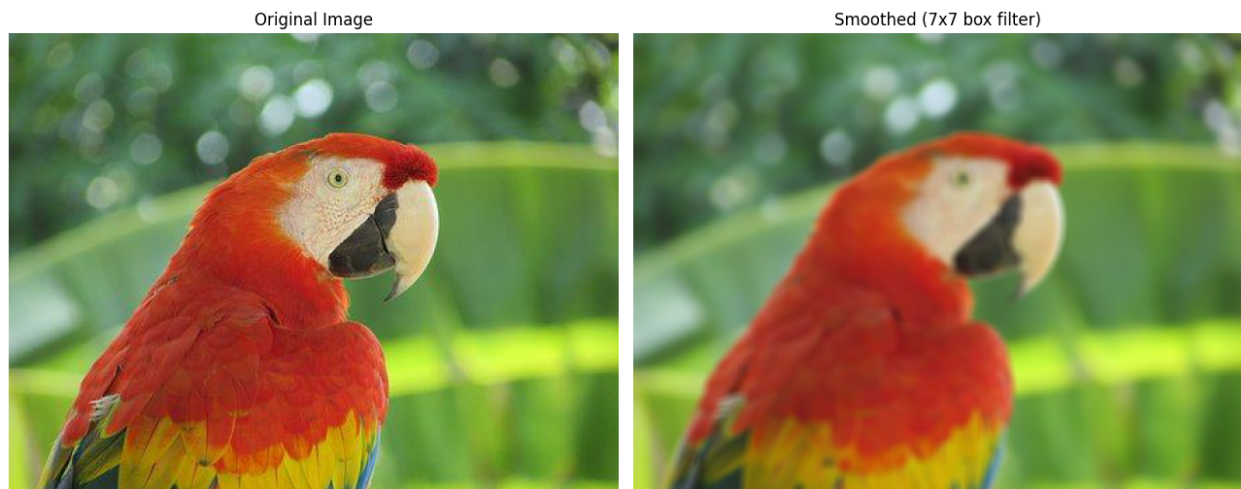


Figure 3 – Task 1: the original and the image smoothed with a 7×7 box filter

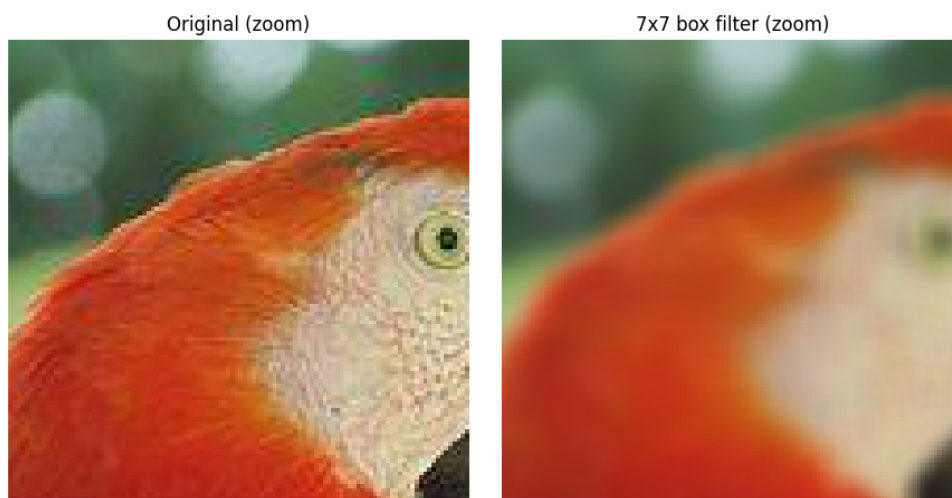


Figure 4 – Task 1: zoom on the eye of the parrot

Gaussian Filters

For the second task I applied `cv2.GaussianBlur()` to the cat with a 5×5 kernel and then with a 21×21 kernel. I left sigma at 0, so OpenCV computed it from the kernel size: 1.1 for the small kernel and 3.5 for the large one.

The 5×5 filter only softened the fur, and the image still looked almost the same from a distance. The 21×21 filter removed the fur completely and left only the large shapes. The variance of the Laplacian went from 399 to 19.7 and then to 2.3, and the mean stayed at 115.3 in all three images.

I also plotted the two kernels. Unlike the box filter, the weights are not equal: the center of the 5×5 kernel holds 13.7% of the total weight, while in the 21×21 kernel the weight is spread over a much larger area and the center holds only 1.3%.



Figure 5 – Task 2: original, 5×5 Gaussian and 21×21 Gaussian



Figure 6 – Task 2: zoom on the eye of the cat

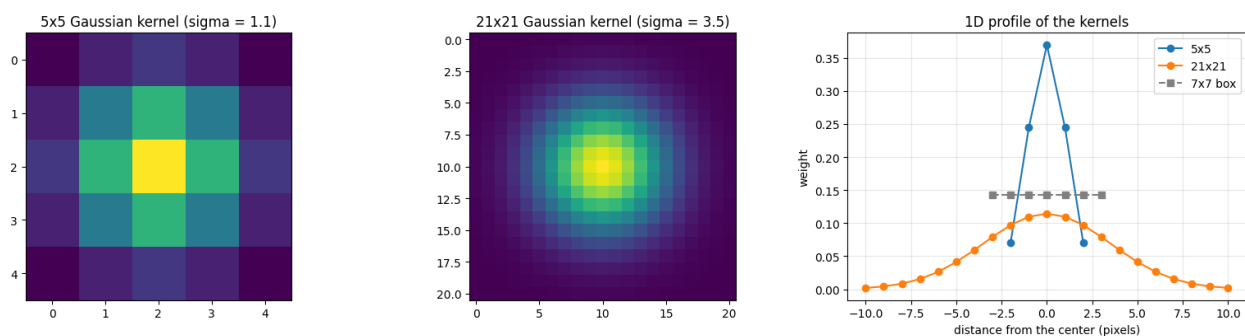


Figure 7 – Task 2: the two Gaussian kernels and their profiles

Laplacian Sharpening

For the third task I followed the steps of the manual on the cameraman: I converted the image to float32, applied `cv2.Laplacian()` with a 3×3 kernel and computed the sharpened image as the original minus the Laplacian, clipped between 0 and 1.

The Laplacian image was gray in the flat regions, such as the sky and the coat, and showed thin dark and bright lines along every edge. Its values went from -4.35 to 3.76, far outside the range of the image, with a mean of zero.

The sharpened image was clearly crisper. The tripod, the camera and the buildings in the background stood out, and the standard deviation rose from 0.289 to 0.357. But the grass became grainy, and 17.7% of the values went outside the range and had to be clipped. A profile along one row showed why: at each edge the sharpened signal overshoots on both sides.

I also filtered a single white pixel to see the kernel that OpenCV really uses. With a size of 3 it is not the classic one with -4 in the center, but a diagonal one with 2 in the four corners and -8 in the center. The classic kernel is what OpenCV uses when the size is 1.

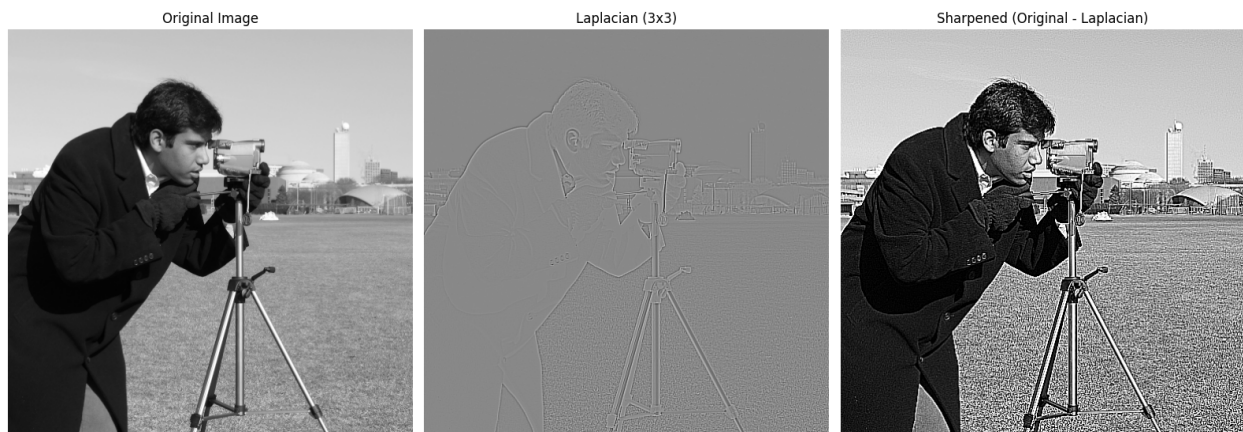


Figure 8 – Task 3: original, Laplacian and sharpened image

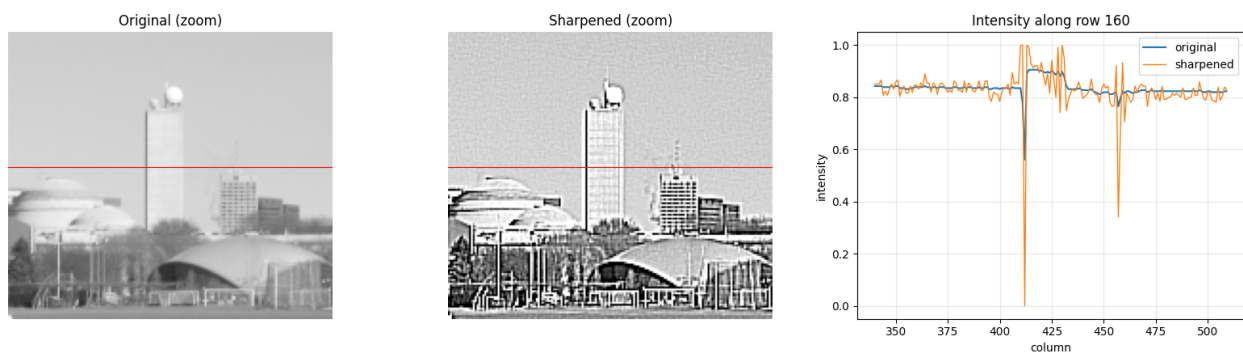


Figure 9 – Task 3: zoom on the buildings and the intensity along one row

Observations

The most surprising thing was that sharpening is built from blurring. Unsharp masking needs a blurred copy of the image to find the detail, and the Laplacian with a negative center does the same job in a single step.

I also noticed that the weights matter as much as the size. The 5×5 Gaussian blurred far less than the 7×7 box filter, not only because it is smaller, but because the box gives the same weight to all its pixels while the Gaussian concentrates the weight in the center.

Finally, sharpening has a cost. It cannot tell detail from noise, so it amplified the grain in the grass together with the edges, and it pushed many pixels outside the valid range.

Conclusion

This lab showed that smoothing and sharpening are opposite operations built on the same idea, a kernel that slides over the image. Smoothing kernels have positive weights that sum to 1, so they average the neighborhood and keep the brightness. The Laplacian has weights that sum to 0, so it responds only where the intensity changes.

The box filter is the simplest way to blur, the Gaussian gives a more natural blur that is controlled by its size and sigma, and unsharp masking and the Laplacian bring the edges back, as long as the amount of sharpening is kept under control.

Submission

The complete work for this lab is in my GitHub repository, in the **Lab5** folder:

<https://github.com/Ali-developer/Image-Processing/tree/main/Lab5>

As required, the Lab5 folder contains the following files:

- Jupyter notebook: Lab5.ipynb, executed, with all the outputs and the task solutions
- Images used: images/Parrot.png, images/camera.png, images/chelsea.png
- This report: Lab5_Report.pdf, and the same write-up as README.md